UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

UML
Use Case Diagram

Francesco Guerra
francesco.guerra@unimore.it

Introduction

- **Purpose**: Capture the **functional requirements** of a system.
- **Approach**: Describe **interactions between users (actors)** and the **system**, as a narrative of how the system is used.
- We employ a use case diagram to answer the following questions:
  1. What is being described? → The system
  2. Who interacts with the system? → The actorsWhat can the actors do? → The use cases
  3. At first glance, this diagram seems to be extremely simple to learn and use.
- The use case diagram is essential for a detailed technical design.
  - It expresses the customer's expectations of the system.
  - The diagram documents the requirements the system should fulfill.

Introduction

- If use cases are forgotten or specified incorrectly, the consequences can be extremely serious:
  - the development and maintenance costs increase,
  - the users are dissatisfied.
- Example of use case
  - The customer browses the catalog and adds desired items to the shopping basket. When the customer wishes to pay, the customer describes the shipping and credit card information and confirms the sale. The system checks the authorization on the credit card and confirms the sale both immediately and with a follow-up e-mail.
    - This is scenario:
      - a sequence of steps describing an interaction between a user and a system. This is one thing that can happen.
    - The credit card authorization might fail.
    - You may have a customer for whom you don't need to capture the shipping and credit card information

Definitions

- Key Steps
  - Identifying Actors
  - Identifying Scenarios
  - Identifying Use Cases
- Actors
  - Definition: External entities that interact with the system.
  - Challenge: Deciding what counts as an actor is tricky and improves with experience.
  - Questions to Identify Actors
    - Which user groups does the system support in performing their work?
    - Which user groups execute the system's main functions?
    - What external hardware or software interacts with the system?
  - Actors represent roles, not specific individuals.
  - A user adopts a role (or multiple roles simultaneously) to perform the associated use cases.

Definitions (2)

- Use cases
  - It describes a functionality expected from the system to be developed.
  - Scope:
    - Encompasses a set of functions executed when using the system.
    - Does not cover internal structure or actual implementation.
  - Key Points
    - Provides tangible benefits for one or more actors interacting with it.
    - Triggered by:
      - Actor invocation, or
      - Trigger events (e.g., semester ends → Issue Certificate use case is executed).
    - Determined by:
      - Collecting customer wishes.
      - Analyzing problems described in natural language, which form the basis of requirements analysis.

Definitions (3)

- Associations
  - Definition: An association connects an actor to a use case, showing that the actor interacts with the system and uses a specific functionality.
  - Key Rules
    - Every actor must be associated with at least one use case.
    - Every use case must be associated with at least one actor.
    - Associations are always binary: one actor ↔ one use case.
  - Multiplicity
    - Can be specified at the ends of the association.
    - Actor end: default is 1 if not specified.
    - Use case end: usually unrestricted, rarely specified explicitly.

Definitions (4)

- Scenarios
  - Definition: A scenario is a narrative description of what people do and experience when interacting with a system.
  - Characteristics:
    - Concrete, focused, and informal.
    - Describes a system feature from the viewpoint of a single actor.
  - Types of Scenarios
    - As-is scenarios – describe the current situation.
    - Visionary scenarios – describe a future system.
    - Evaluation scenarios – describe tasks for system evaluation.
    - Training scenarios – tutorials for introducing new users to the system.
  - Questions to Identify Scenarios
    - What tasks does the actor want the system to perform?
    - What information does the actor access?
    - Which external changes does the actor need to inform the system about?

How to write a Use Case diagram

Buy a Product

Goal Level: Sea Level

Main Success Scenario:

1. Customer browses catalog and selects items to buy
2. Customer goes to check out
3. Customer fills in shipping information (address; next-day or 3-day delivery)
4. System presents full pricing information, including shipping
5. Customer fills in credit card information
6. System authorizes purchase
7. System confirms sale immediately
8. System sends confirming e-mail to customer

Extensions:

3a: Customer is regular customer

.1: System displays current shipping, pricing, and billing information

.2: Customer may accept or override these defaults, returns to MSS at step 6

6a: System fails to authorize credit purchase

.1: Customer may reenter credit card information or may cancel

- You begin by picking one of the scenarios as the main success scenario.
- You start the body of the use case by writing the main success scenario as a sequence of numbered steps.
- You then take the other scenarios and write them as extensions, describing them in terms of variations on the main success scenario.

Content of a Use Case

- Primary Actor
  - Each use case has a primary actor: the actor whose goal the use case aims to satisfy.
  - Usually, the initiator of the use case.
- Steps
  - Represent interactions between the actor and the system.
  - Each step is a simple statement showing who performs it.
  - Emphasizes the intent of the actor, not the internal mechanics.
- Extensions
  - Describe conditions that lead to alternative interactions.
  - Specify how these interactions differ from the main scenario.
- Included Use Cases
  - A complicated step can itself be another use case.
  - UML term: the first use case includes the second.
  - No standard way to represent included use cases in text (hyperlinks may be used).

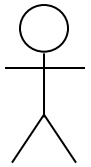
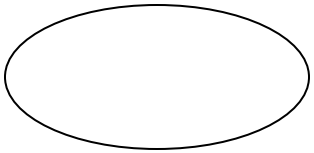
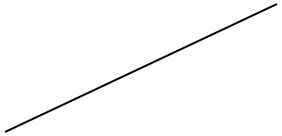
Content of a Use Case
Additional Information

- Pre-condition
  - Specifies what the system ensures is true before the use case begins.
  - Helps programmers know which conditions do not need to be checked in code.
- Guarantee (Post-condition)
  - Describes what the system ensures after the use case ends.
- Trigger
  - The event that starts the use case.
- Guidelines
  - Be skeptical when adding extra elements: too much detail can be counterproductive.
  - The level of detail should reflect the risk associated with the use case.

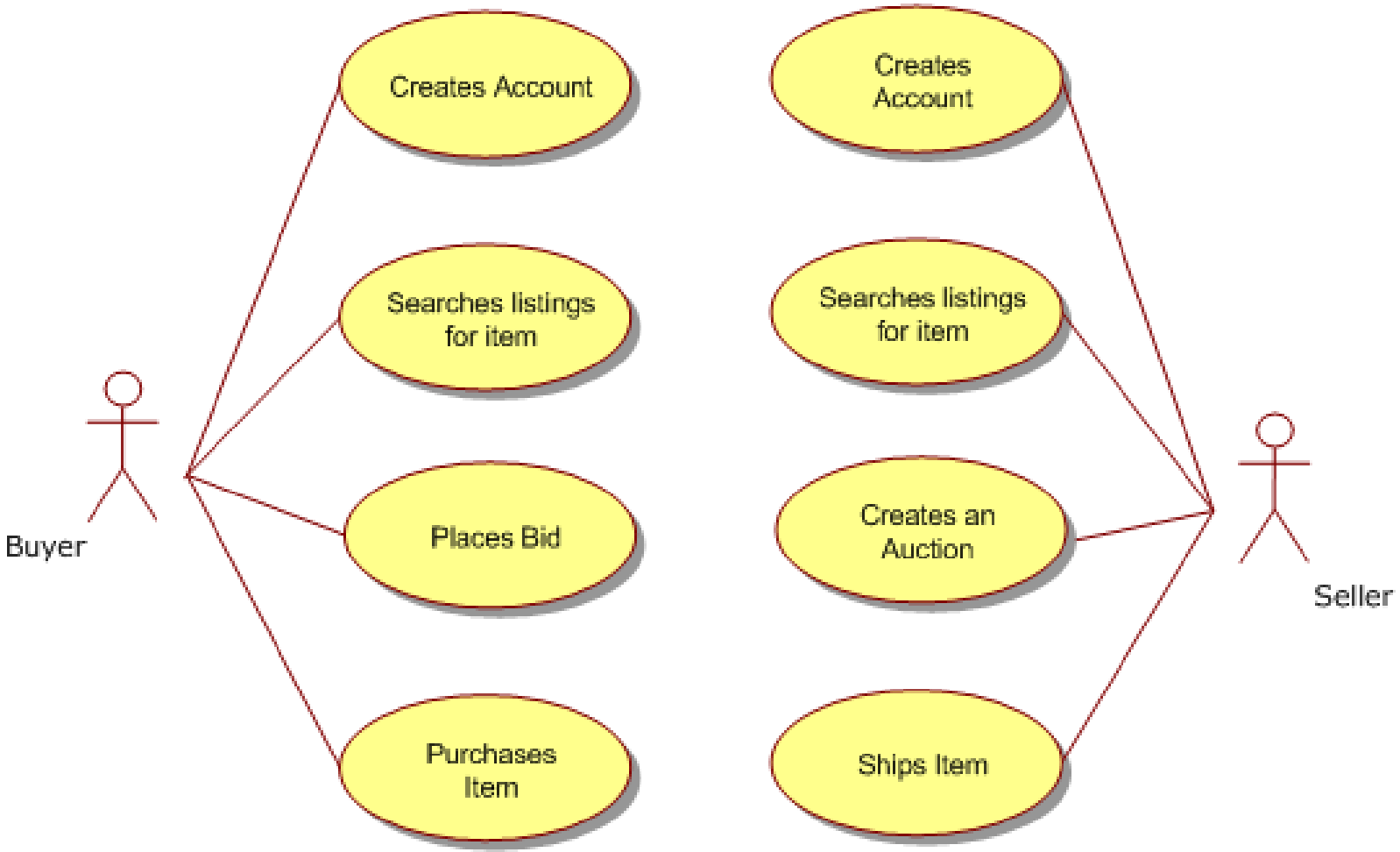
Use Case Diagrams

- The best way to think of a use case diagram is that it's a graphical table of contents for the use case set.
  - It shows the system boundary and the interactions with the outside world.
- The use case diagram shows the actors, the use cases, and the relationships between them:
  - Which actors carry out which use cases
  - Which use cases include other use cases
- The UML includes relationships between use cases
  - Inheritance
  - Includes
  - Extends

Use Case - Elements

	Actor
	Use-case
	Relationship
	System Boundary

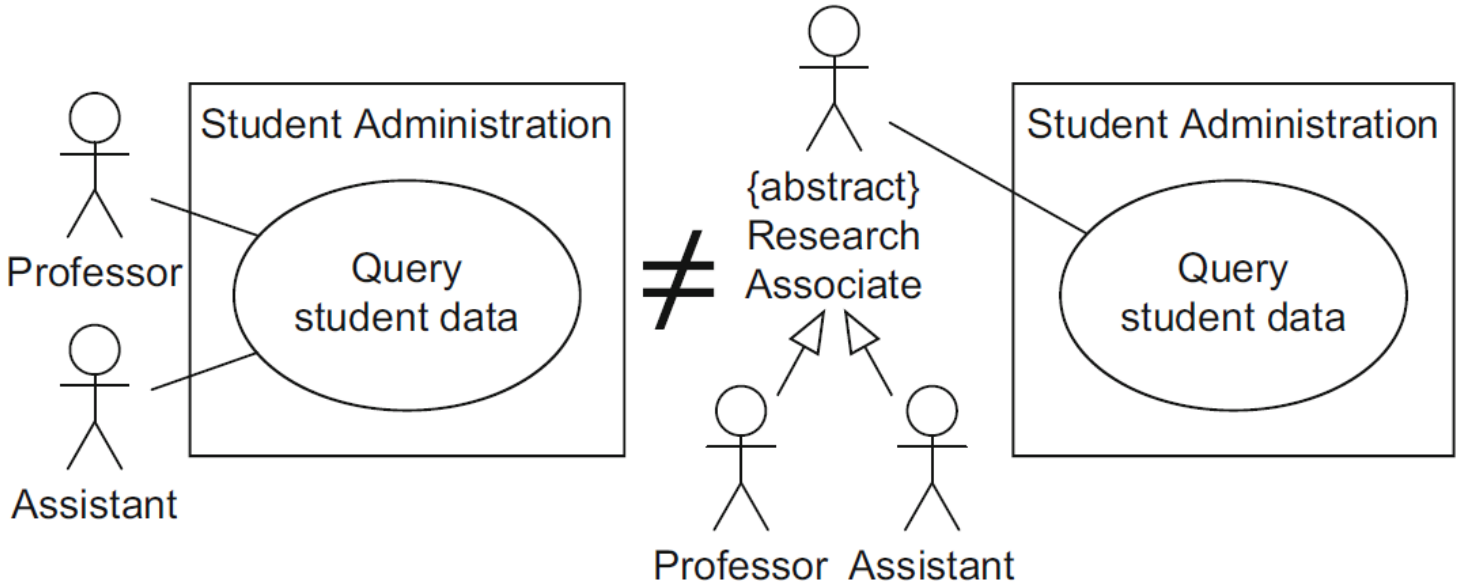
Example


Creates Account
Searches listings for item
Places Bid
Purchases Item
Creates Account
Searches listings for item
Creates an Auction
Ships Item
Buyer
Seller

Inheritance

- Actors often have common properties and some use cases can be used by various actors.
  - For example, it is possible that not only professors but also other research personnel are permitted to view student data.
- To express this, actors may be depicted in an *inheritance relationship* (generalization) with one another.
  - When an actor Y (sub-actor) inherits from an actor X (super-actor), Y is involved with all use cases with which X is involved.
  - In simple terms, generalization expresses an “is a” relationship.

Example

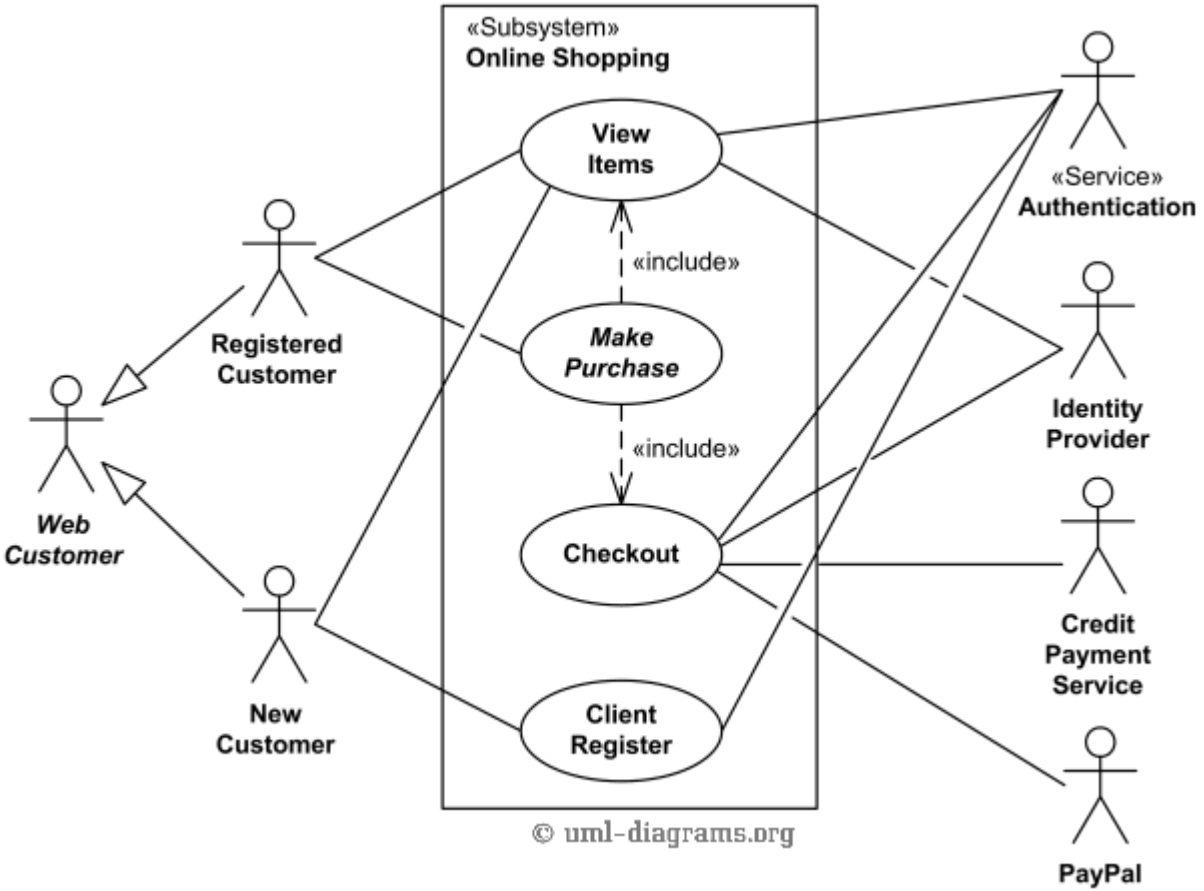


- There is a difference between two actors participating in a use case themselves and two actors having a common super-actor that participates in the use case.
- If there is no instance of an actor, this actor can be labeled with the keyword {abstract}, or the label has to be written in italic.

Included relationship

- If a use case A includes a use case B, the behavior of B is integrated into the behavior of A.
  - A is referred to as the *base use case*.
    - The base use case always requires the behavior of the included use case to be able to offer its functionality.
  - B as the *included use case*.
    - The included use case can be executed on its own.
- The use of «include» is analogous to calling a subroutine in a procedural programming language.
- One use case may include multiple other use cases.
- One use case may also be included by multiple different use cases.
  - In such situations, it is important to ensure that no cycle arises.

Example


«Subsystem»
Online Shopping
View Items
«include»
Make Purchase
«include»
Checkout
Client Register
Registered Customer
Web Customer
New Customer
«Service»
Authentication
Identity Provider
Credit Payment Service
PayPal
© uml-diagrams.org

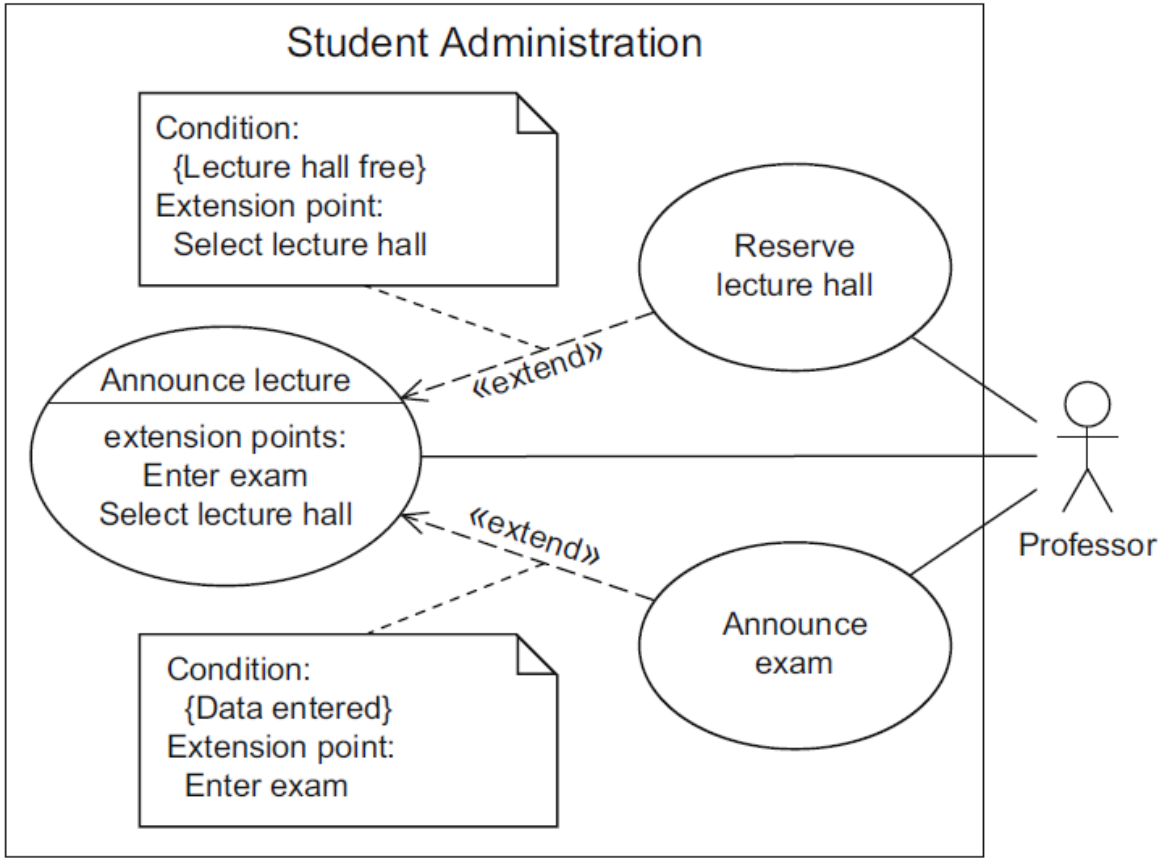
Extended relationship

- If a use case B is in an «extend» relationship with a use case A, then A can use the behavior of B but does not have to.
- Use case B can therefore be activated by A in order to insert the behavior of B in A.
- Here A is again referred to as the *base use case* and B as the *extending use case*.
- Both use cases can also be executed independently of one another.
- A use case can act as an extending use case several times or can itself be extended by several use cases.
- Again, no cycles may arise. Note that the arrow indicating an «extend» relationship points towards the base use case, whereas the arrow indicating an «include» relationship originates from the base use case and points towards the included use case.

Extended relationship

- A *condition* that must be fulfilled for the base use case to insert the behavior of the extending use case can be specified for every «extend» relationship.
  - The condition is specified, within curly brackets, in a note that is connected with the corresponding «extend» relationship.
  - A condition is indicated by the preceding keyword Condition followed by a colon.
- By using *extension points*, you can define the point at which the behavior of the extending use cases must be inserted in the base use case.

Example


Student Administration
Condition:
{Lecture hall free}
Extension point:
Select lecture hall
Reserve lecture hall
«extend»
Announce lecture
extension points:
Enter exam
Select lecture hall
«extend»
Professor
Announce exam
Condition:
{Data entered}
Extension point:
Enter exam

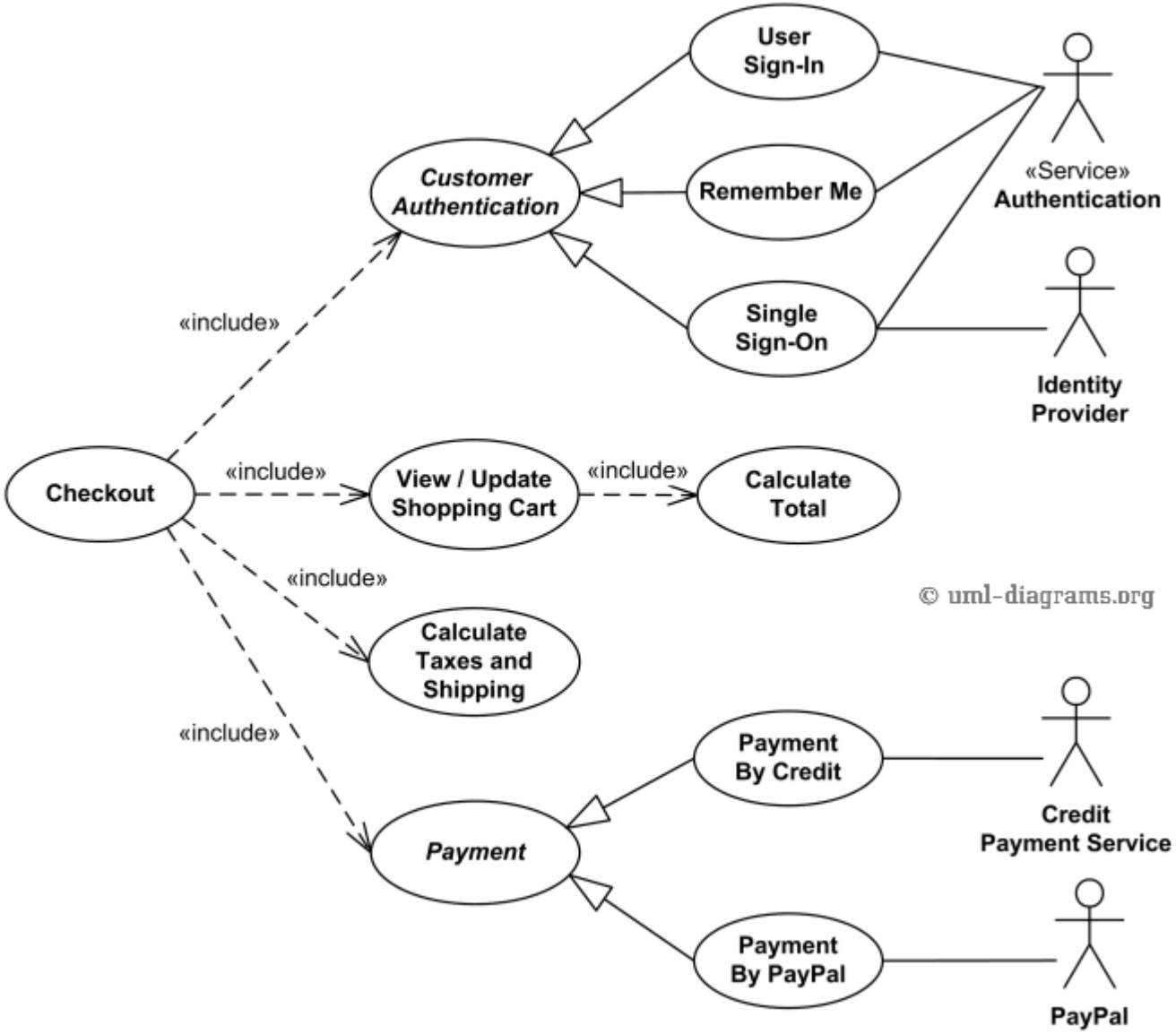
UML Definitions

- **Extends**
  - *An Extend is a relationship **from an extending UseCase** (the extension) **to an extended UseCase** (the extendedCase) that specifies how and when the behavior defined in the extending UseCase can be inserted into the behavior defined in the extended UseCase. The extension takes place at one or more specific extension points defined in the extended UseCase.*
  - *Extend is intended to be used when there is some additional behavior that should be added, **possibly conditionally**, to the behavior defined in one or more UseCases.*
  - *The **extended UseCase** is defined independently of the extending UseCase and **is meaningful independently of the extending UseCase**. On the other hand, the **extending UseCase** typically defines behavior that **may not necessarily be meaningful by itself**. Instead, the extending UseCase defines a set of modular behavior increments that augment an execution of the extended UseCase under specific conditions.*
- **Includes**
  - *Include is a DirectedRelationship between two UseCases, indicating that the behavior of the **included UseCase** (the addition) **is inserted into the behavior of the including UseCase** (the includingCase). It is also a kind of NamedElement so that it can have a name in the context of its owning UseCase (the includingCase). The including UseCase may depend on the changes produced by executing the included UseCase. The included UseCase must be available for the behavior of the including UseCase to be completely described.*
  - *The Include relationship is intended to be used when there are common parts of the behavior of two or more UseCases. This **common part is then extracted to a separate UseCase, to be included by all the base UseCases having this part in common**. As the primary use of the Include relationship is for reuse of common parts, what is left in a **base UseCase is usually not complete in itself** but dependent on the included parts to be meaningful. This is reflected in the direction of the relationship, indicating that the base UseCase depends on the addition but not vice versa.*

Example (2)


Search Items
«extend»
Browse Items
«extend»
«extend»
View Items
View Recommended Items
«include»
«extend»
Add to Shopping Cart
«extend»
Customer Authentication
«extend»
Add to Wish List
«include»
© uml-diagrams.org

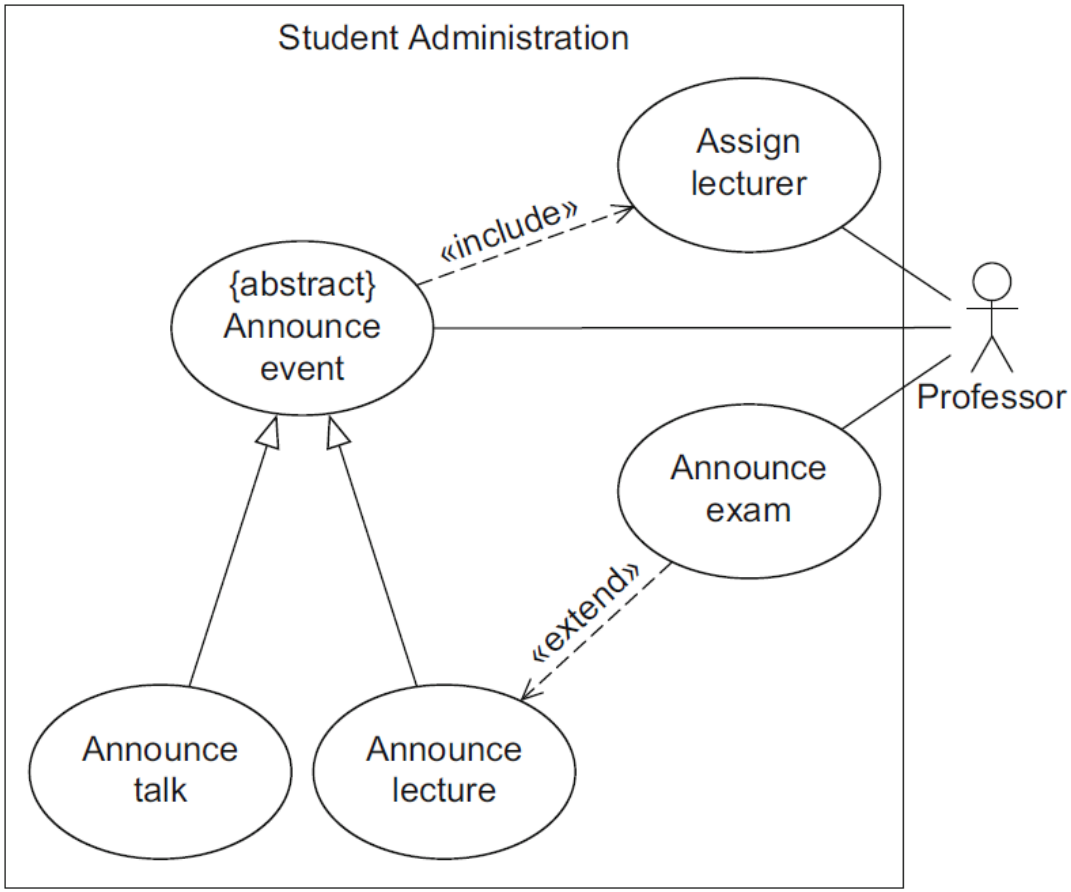
Example (3)


User Sign-In
Customer Authentication
Remember Me
Single Sign-On
«Service» Authentication
Identity Provider
«include»
Checkout
«include»
View / Update Shopping Cart
«include»
Calculate Total
«include»
Calculate Taxes and Shipping
«include»
Payment
Payment By Credit
Credit Payment Service
Payment By PayPal
PayPal
© uml-diagrams.org

Generalization for use cases

- *Generalization* is also possible between use cases.
  - Common properties and common behavior of different use cases can be grouped in a parent use case.
  - If a use case A generalizes a use case B, B inherits the behavior of A, which B can either extend or overwrite. Then, B also inherits all relationships from A.
  - B adopts the basic functionality of A but decides itself what part of A is executed or changed.
- If a use case is labeled {abstract}, it cannot be executed directly; only the specific use cases that inherit from the abstract use case are executable.

Example


Student Administration
Assign lecturer
«include»
{abstract} Announce event
Professor
Announce exam
«extend»
Announce talk
Announce lecture

Levels of Use Cases

- Problem
  - System use case: interaction with the software.
  - Business use case: describes how a business responds to a customer or event.
- Use Case Levels
  - Sea-level
    - Usually system use cases representing discrete interactions between a primary actor and the system.
    - Deliver value to the primary actor.
  - Fish-level
    - Use cases included by sea-level use cases.
    - Typically represent supporting interactions within the system.
  - Kite-level
    - Show how sea-level use cases fit into wider business interactions.
    - Usually business use cases, focusing on the process rather than the software alone.

Use Cases and Features (or Stories)

- **Features**
  - Represent **functional elements** of a system (called **user stories** in **Extreme Programming**).
  - Useful for **planning iterative projects**, where each iteration delivers a set of features.
- **Use Cases**
  - Provide a **narrative** of how actors interact with the system.
  - Focus on **"who does what"** and **why**, rather than small implementation details.
- **Relationship Between Features and Use Cases**
- Features and use cases are **closely related**:
  - A feature can correspond to:
    - **A whole use case**
    - **A scenario within a use case**
    - **A step in a use case**
- Typically, **features are more fine-grained** than use cases.
- Recommended approach: develop **use cases first**, then extract a list of **features**.

When to use Use Cases

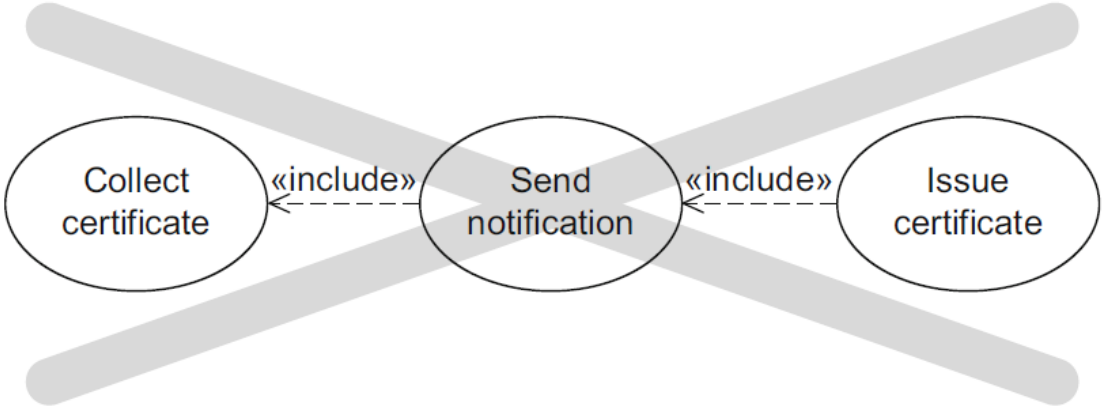
- Use cases
  - are a valuable tool to help understand the functional requirements of a system.
  - represent an external view of the system.
    - No correlations between use cases and the classes inside the system.
- With use cases, concentrate your energy on their text rather than on the diagram.
  - Despite the fact that the UML has nothing to say about the use case text, it is the text that contains all the value in the technique.
- A big danger of use cases is that people make them too complicated.

Describing Use Cases

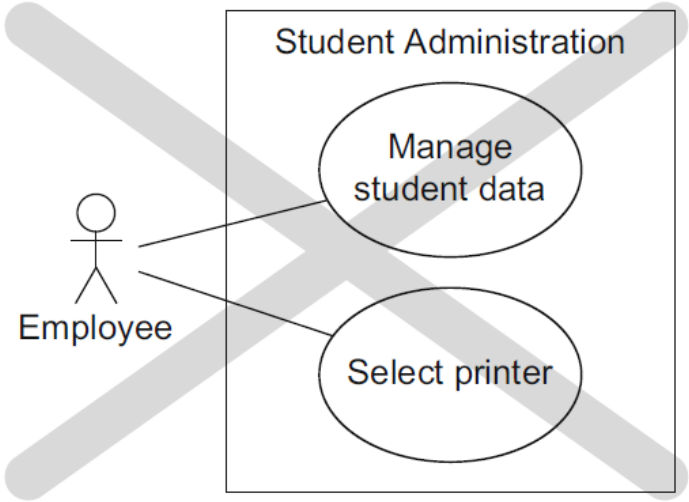
- To ensure that large use case diagrams remain clear, it is important to select concise names for the use cases.
- When this is not possible, you must describe the use cases.
  - You have to describe the use cases clearly and concisely, as otherwise there is a risk that readers will only skim over the document.
- A structured approach for the description of use cases can contain:
  - Name
  - Short description
  - Precondition: prerequisite for successful execution
  - Postcondition: system state after successful execution
  - Error situations: errors relevant to the problem domain
  - System state on the occurrence of an error
  - Actors that communicate with the use case
  - Trigger: events which initiate/start the use case
  - Standard process: individual steps to be taken
  - Alternative processes: deviations from the standard process

Pitfalls

- *Error 1: Modeling processes*

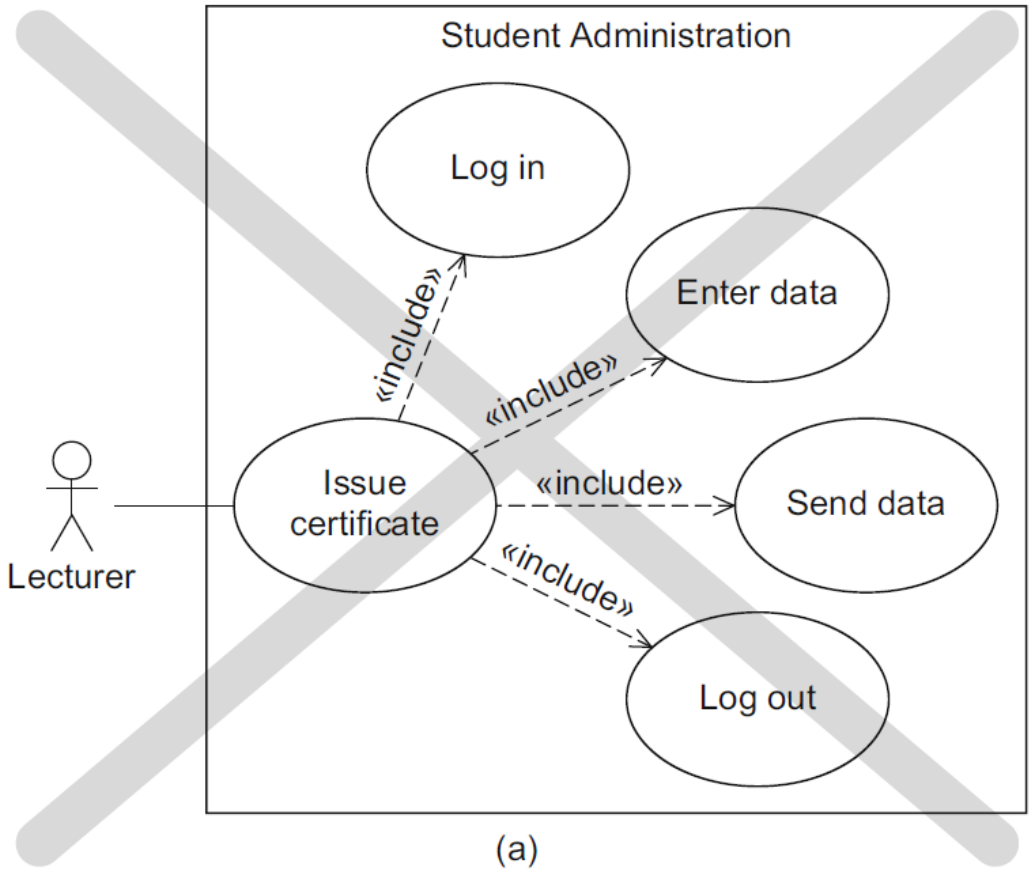


- *Error 2: Mixing abstraction levels*

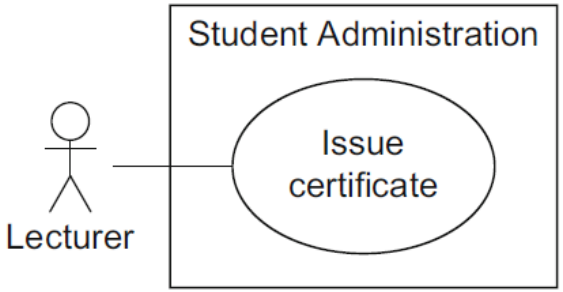


Pitfalls

- *Error 3: Functional decomposition*

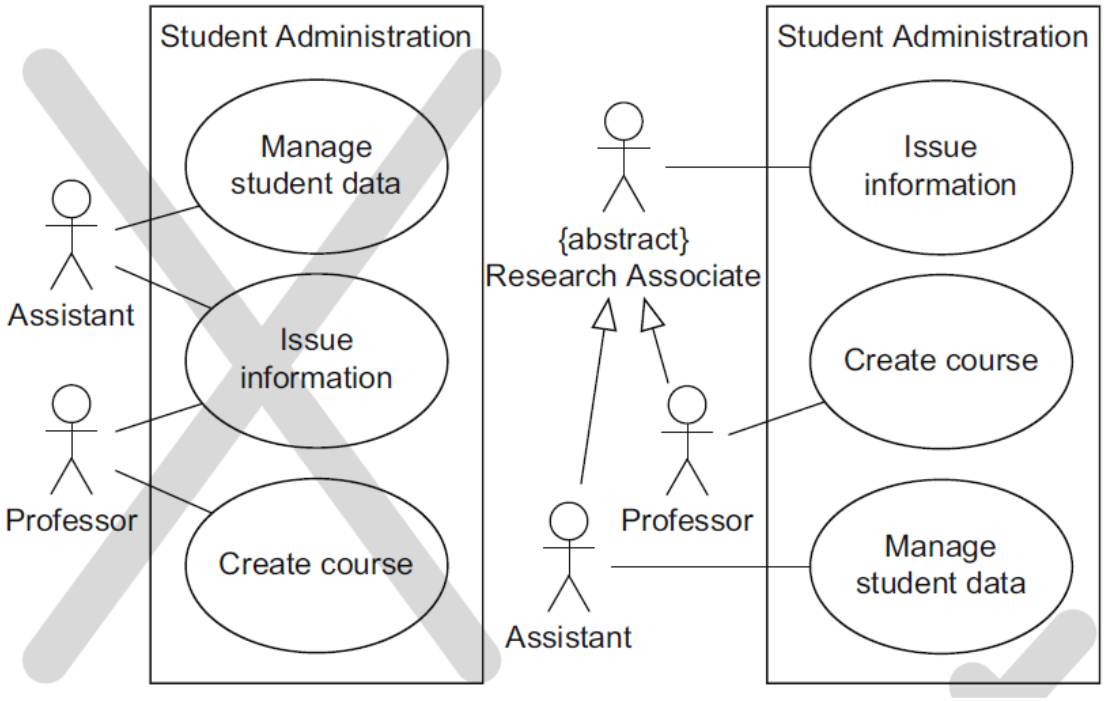


(a)



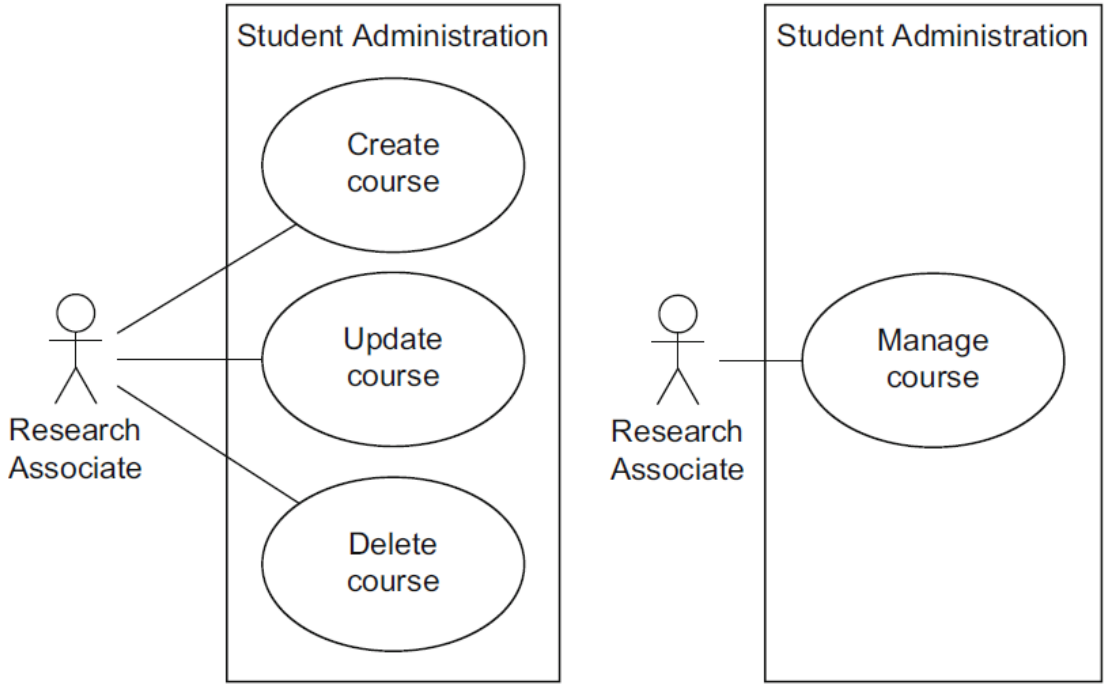
Pitfalls

- *Error 4: Incorrect associations*



Pitfalls

- *Error 5: modeling redundant use cases*



Conclusion

- The use case structure is a great way to brainstorm alternatives to the main success scenario.
  - For each step, ask, How could this go differently?
  - What could go wrong?
  - It's usually best to brainstorm all the extension conditions first, before you get bogged down working out the consequences.

Esercizio (un problema reale) ;-)

- Si descriva in forma testuale e con il corrispondente formalismo grafico il processo di acquisto di una bevanda dal distributore automatico. L'utente può decidere di pagare o utilizzando monete o utilizzando una chiavetta.
- Si rappresenti il caso d'uso utilizzando un elenco di passi e identificando anche possibili estensioni per gestire situazioni particolari (il prodotto non è disponibile, l'importo non è sufficiente, le monete non sono conformi …)

Esercizio

- Un'agenzia di viaggi fornisce ai propri clienti servizi di prenotazione per: camere d'albergo, biglietti per treni e aerei, automobili e visite guidate. L'agenzia vende anche pacchetti turistici completi, creati assemblando opportunamente I singoli servizi elencati in precedenza.
- Prima di potere accedere a qualsiasi servizio, il cliente deve essere opportunamente registrato e ogni pagamento deve avvenire entro 30 giorni dall'acquisto. I clienti morosi devono ricevere un sollecito.

Esercizio

- I dipendenti di una officina hanno il compito di registrare i dati dei veicoli in ingresso, incluso i dati dei proprietari dei veicoli, di interrograre l'archivio delle riparazioni da effettuare, e di consegnare o veicoli riparati ai clienti,
- In caso di riparazioni particolarmente complesse, la consegna viene fatta dai direttori che rilasciano una particolare garanzia ai clienti. I direttori possono interrogare e modificare i dati personali dei dipendenti.
- L'ufficio marketing si occupa delle comunicazioni ai clienti e deve quindi accedere i dati dei clienti.

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

References

- Martin Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language , Pearson Addison Wesley
- Russ Miles, Kim Hamilton: Learning UML 2.0- A Pragmatic Introduction to UML, O'Reilly Media, 2006
- Martina Seidl, Marion Scholz, Christian Huemer, Gerti Kappel: UML @ Classroom - An Introduction to Object-Oriented Modeling, Springer 2015

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.